R4.01 Architecture logicielle

CM1 : JS et nodeJS

PN

Descriptif

L’objectif de cette ressource est de présenter des composants de la programmation qui peuvent être utilisés dans plusieurs domaines.

Savoirs de référence étudiés

- Patrons d’architecture (par ex. : Modèle Vue Contrôleur, Model View ViewModel), ...)
- Utilisation de briques logicielles, d’interfaces de programmation, de bibliothèques tierces
- Développement de services web

Objectif

Développer des microservices RestFull

CM : 8 / TD : 25

JavaScript / NodeJS / MongoDB / Express
WebSocket

Une évaluation finale, une évaluation intermédiaire

JavaScript : ES6 (ES2015)

- Langage orienté objet
- Interprété
- Faiblement typé
- Exécuté principalement coté client mais aussi serveur nodeJS

Variables et types

const et let pour introduire des constante (attention aux références) et des variables, il faut éviter var ou les déclarations globales.

La protée avec let, const ou var et le bloc.

Préférer le mode strict.

```
"use strict"
const pi  = 3.1415;
console.log(pi);
const circonference = function(rayon){
   const pi = 3.14159; //possible
   return 2*pi*rayon;
}
```

Que donne

pi = 3.1416;

a=10 ;

Var a = 44;

Variables et types

Pourquoi éviter var ?

```
const fn=function() {
    console.log(foo); // undefined (au lieu de ReferenceError)
    var foo = "bar";
}

fn();
```

Variables et types

Primitifs

- Booléen :true ou false
- Null
- Undefined : lorsqu'une valeur n'a pas été définie
- Nombre
- Chaîne de caractères
- Symboles

Objets : Object, Date, Array, String, Math, RegExpr, …

Chaines de caractères

Entre "" ou ‘’ mais aussi `` pour les substitutions :

```
const s1 ="abc"
const s2 ='de'
const s3 = `chaine1 : ${s1} chaine2 : ${s2}`;

console.log(s3);
//chaine1 : abc chaine2 : de
```

Objet littéral

```
"use strict"
const robert = {
    poids: 45,
    taille : 180,
    couleur : 'blue',
    aboie : function () {
        console.log("woawoa");
    }
}

console.log(robert);
//{ poids: 45,
//    taille: 180,
//    couleur: 'blue',
//    aboie: [Function: aboie] }
robert.aboie; //rien
robert.aboie(); //woawoa
```

Objet littéral

```
"use strict"
const robert = {
    poids: 45,
    taille : 180,
    couleur : 'blue',
    aboie : function () {
        console.log("woawoa");
    }
}
```

Je change robert.

```
robert.aboie = false;
robert. sexe = "garcon";

console.log(robert);
```

Bilan, j'ai modifié un attribut (une propriété) qui est passé d'une fonction à un booléen et je l'ai également augmenté robert d'un nouvel attribut.

Object

Le type Object est l'un des types de données JavaScript.

Il est utilisé pour stocker des ensembles de clés/valeurs et des entités plus complexes.

Les objets peuvent être créés avec le constructeur Object()

Object

Le constructeur Object crée une enveloppe objet pour la valeur passée en argument.

Object()

```
let o = new Object();
o.toto = 42;
console.log(o);
console.log(Object.getPrototypeOf(o))
```

{ toto: 42 }

[Object: null prototype] {}

Objet Litteral

Constructeur Object :

Object.create()

Crée un nouvel objet avec le prototype d'objet et les propriétés indiqués.

```
const person = {
    isHuman: false,
    printIntroduction: function () {
        console.log(`My name is ${this.name}.
Am I human? ${this.isHuman}`);
    }
};

const me = Object.create(person);

me.name = "Matthew";

// "name" is a property set on "me", but
not on "person"
me.isHuman = true;

// inherited properties can be overwritten

me.printIntroduction();
```

Object

Constructeur Object :

Object.assign()

Copie les valeurs de toutes les propriétés propres énumérables depuis un ou plusieurs objets source dans un objet cible.

```
const target = { a: 1, b: 2 };
const source = { b: 4, c: 5 };

const returnedTarget =
Object.assign(target, source);

console.log(target);
// expected output: Object { a: 1, b:
4, c: 5 }

console.log(returnedTarget);
// expected output: Object { a: 1, b:
4, c: 5 }
```

Objet Litteral

Constructeur Object :

Object.entries()

```
const object1 = {
  a: 'somestring',
  b: 42
};

for (let [key, value] of
Object.entries(object1)) {
  console.log(`${key}: ${value}`);
}

// expected output:
// "a: somestring"
// "b: 42"
// order is not guaranteed
```

Objet Litteral

Constructeur Object :

Object.fromEntries()

Renvoie un nouvel objet à partir d'un itérable contenant des paires de clés-valeurs

```
const entries = new Map([
    ['foo', 'bar'],
    ['baz', 42]
]);

const obj =
Object.fromEntries(entries);

console.log(obj);
// expected output: Object { foo:
"bar", baz: 42 }
```

Objet Litteral

Constructeur Object :

Object.keys()

Retourne un tableau contenant les noms de toutes les propriétés énumérables propres de l'objet donné.

```
const object1 = {
    a: 'somestring',
    b: 42,
    c: false
};

console.log(Object.keys(object1));
// expected output: Array ["a",
"b", "c"]
```

Objet Litteral

Constructeur Object :

Object.values()

Retourne le tableau des valeurs énumérables propres de l'objet donné dont les clés sont des chaînes de caractères.

```
const object1 = {
    a: 'somestring',
    b: 42,
    c: false
};

console.log(Object.values(object1
));
// expected output: Array
["somestring", 42, false]
```

Opérateur : décomposition …

… permet d'extraire des données d'un tableau ou d'un objet

```
[a, b] = [10, 20];

console.log(a);
// expected output: 10

console.log(b);
// expected output: 20

[a, b, ...rest] = [10, 20, 30, 40, 50];

console.log(rest);
// expected output: Array [30,40,50]
```

Fonctions

- Une fonction est une variable

```
function maFonction(arg1, arg2=val1)
{ // code
  return valeur;
}
```

ou

```
Const  maVar = function(arg1, arg2)
{
      // code
            return valeur;
}
```

- Avantage redéfinition possible en modifiant la variable, tableaux de fonctions, …
- Rem : Auto exécution avec () à la fin pour la forme let f = function(){}();

Que donne

```
const test1= function() {
    const x = 1;
    test2 = function() {
        const x = 2;
        test3 = function() {
            const x = 3;
            console.log(x);
        }();
    }();
    console.log(x);}();
console.log(x);
```

Array

- forEach
- map
- filter
- Reduce
- …

POO en JS

- Rappels des similitudes avec PHP et Kotlin
  - Classe : c'est un modèle, une définition d'une "structure", qui va posséder des attributs et des méthodes.
  - Instance : c'est un exemplaire d'une classe.
  - Attribut : c'est une variable qui "appartient" à un objet.
  - Méthode : c'est une fonction propre à une classe. Elle va pouvoir agir sur l'instance à laquelle on l'applique.
- Une différence
  - Objet : selon le sens, il s'agit d'une classe ou d'une instance.
- Utilisation
  - Instanciation new
  - Accès à un attribut .nomAttribut ou [nomAttribut] (Array est un objet)
  - Accès méthode nomObjet.laMethode()
  - Destruction en affectant null

POO et prototype

- Non avons vu qu'un objet peut-être enrichie à l'exécution mais comme un objet peut-être vu comme une classe, il est possible grâce à prototype d'enrichir ses instances.

- La programmation orientée prototype est une forme de programmation orientée objet sans classe, basée sur la notion de prototype. Un prototype est un objet à partir duquel on crée de nouveaux objets.

Depuis ES5, il est également possible d'utiliser des classes.

POO et prototype

- Un principe une fonction n’est pas seulement une variable mais c’est aussi un objet
- this permet d’accèder aux membres

```
// Déclare une fonction constructeur
const Rectangle = function (largeur, hauteur) {
    this.largeur = largeur; // sauve la largeur dans l'instance
    this.hauteur = hauteur; // sauve la hauteur dans l'instance
}

const r1 = new Rectangle(10, 4); // crée un instance
const r2 = new Rectangle(12, 8); // crée une autre instance
```

POO et prototype

Nous avons des méthodes : comment ajouter des méthodes ?

En étendant le prototype :

```
Rectangle.prototype.surface = function() {
  return this.largeur * this.hauteur;
}
```

```
console.log(Object.getPrototypeOf(r1))
//{ surface: [Function (anonymous)] }
```

- Rem le prototype permet aussi de rajouter des attributs

POO et prototype

Les attribut propre ne sont pas dans le prototype

```
const Rectangle = function (largeur, hauteur) {
    this.largeur = largeur; // sauve la largeur dans l'instance
    this.hauteur = hauteur; // sauve la hauteur dans l'instance

    surface = function() {
        return this.largeur * this.hauteur;
    }
}
```

```
const r1 = new Rectangle(10,20)

console.log(Object.getPrototypeOf(r1))
//{}
```

Poo et prototype

- Que donne le code suivant

```
const Test = function () {
    this.abc = 444;
}
const a = new Test();
const b = new Test();
Test.prototype.toto = 12;
a.titi = 4;
console.log(a.toto);
console.log(a.titi);
console.log(b.toto);
console.log(b.titi);
```

POO et prototype

- Call et Apply : une fonction javascript qui peut "appartenir" à plusieurs objets. Et donc être appelée dans des contextes d'instances différents, on peu utiliser apply et call pour réaliser ça.

- une_fonction.apply(instance, tableau);
- une_fonction.call(instance, p1, p2, p3, ...);

POO et prototype

```
const getHtmlColor = function () {
    return "#" + this.couleur;
}
const Voiture = function
(codeCouleur) {
    this.couleur = codeCouleur;
}
const Rectangle = function (larg,
haut, rgb) {
    this.couleur = rgb;
    this.largeur = larg;
    this.hauteur = haut;
}
```

```
const v = new Voiture('red');
const r = new Rectangle(10, 12,
'noir');
console.log(getHtmlColor.apply(v));
// affiche #red
console.log( getHtmlColor.apply(r) );
// affiche #noir
//console.log(v.getHtmlColor()); n'a
pas de sens
```

POO et prototype

Pour la culture : Héritage sans extends

```
const Personne = function (prenom, nom) {
   this.type = "Personne";
   this.prenom = prenom || "John";
   this.nom = nom || "Doe";
}


Personne.prototype.getType = function () {
   return this.type;
}


Personne.prototype.getNomComplet = function () {
   return this.prenom + " " + this.nom;
}


const Etudiant = function (prenom, nom, session) {
   Personne.call(this, prenom, nom);
   this.type = "Étudiant session : " + session;
}


Etudiant.prototype = new Personne(); // Héritage se fait ici

Etudiant.prototype.constructor = Etudiant; //chainage des constructeurs
```

```
const e = new Etudiant("Paul","Personne",2018);
console.log(e);
console.log(Etudiant);
console.log(Etudiant.prototype);

Etudiant {
   type: 'Étudiant session : 2018',
      prenom: 'Paul',
      nom: 'Personne'

}

 [Function: Etudiant]

   Etudiant {
      type: 'Personne',
         prenom: 'John',
         nom: 'Doe',
         constructor: [Function: Etudiant] }
```

POO et Prototype

```
class Forme {
    constructor(x, y) {
        this.x = 0;
        this.y = 0;
    }

    deplacer(x, y) {
        this.x += x;
        this.y += y;
        console.info('Forme déplacée.' + x + " " + y);
    }
}

class Rectangle extends Forme {
    constructor(x) {
        super(x, x);
    }
}
```

POO et prototype

- Possibilité d’imbriquer des fonctions, la fonction imbriquée accède au contexte de la fonction englobante, on parle de closure.

- Nous avons vu qu’une fonction a un this, comment depuis une fonction accéder au this de la fonction englobante ?

=> Utiliser les fonctions fléchées

Fonction fléchées : () => {}

```
class Humain {
   constructor(sex,age){
      this.sex=sex;
      this.age=age;
   }
   toString(){
      const mensonge = function(){
         console.log(this);
         //probleme le this n'est pas celui de Humain mais celui de
mensonge
         return 20;
      }
      return "Humain "+this.sex+" "+mensonge();
   }
}

const robert = new Humain("F",18);
console.log(robert.toString());
//undefined
//Humain F 20
```

```
//Une version avec une fonction flechee qui n"a pas this
class Humain2 {
   constructor(sex,age){
      this.sex=sex;
      this.age=age;
   }
   toString(){
      const mensonge = () =>{
         console.log(this);
         //le this est celui de Humain2, mensonge n'en a pas
         this.age -=10;
         return this.age;
      }
      return "Humain "+this.sex+" "+mensonge();
   }
}
const robert2 = new Humain2("F",18);
console.log(robert2.toString());
console.log(robert2.age);
//Humain2 { sex: 'F', age: 18 }
//Humain F 8
//8
```

Les espaces de nommage

- Pas de visibilité par tous : depuis ES7 (private #)

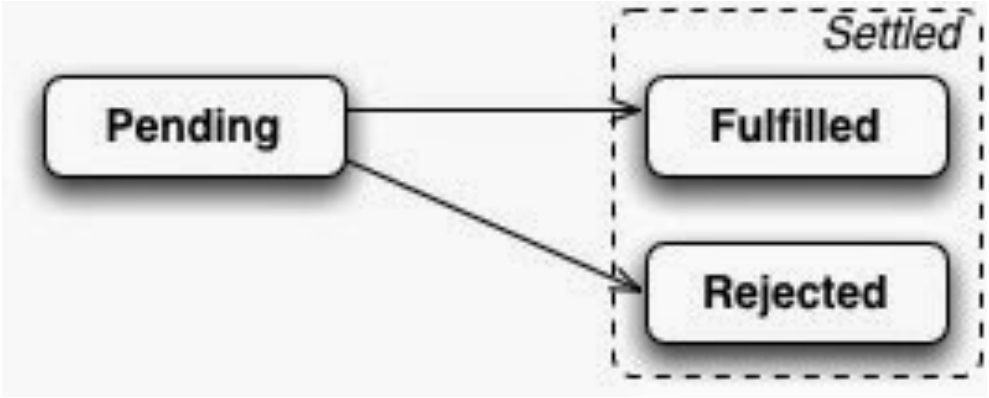
- Pas de package mais simple à simuler

```
const com = {};
com.exemple = {};

com.exemple.app = function(name, url) {
  this._name = name;
  this._url = url;
}

new com.exemple.app("foo", "foo.com");
```

Les promesses : définition



- Une promesse est un objet dont le résultat n'est pas encore connu

```
const unePromesse =new
Promise(function(resolve,reject) {
        resolve(value); // OK

        reject(error); //pb

});
```

Les promesses utilisation

```
maPromesse.then(value => { /* OK */ })
maPromesse.catch(error => { /* Probleme */ });
```

then et catch retournent une promesse =>
les promesses peuvent-être cascadées en séquence : maPromesse.then().then().catch()

```
Promise.all([maPromesse1, maPromesse2]).then()
.catch(); //Synchro
```

```
Promise.raise([maPromesse1, maPromesse2]).then()
.catch(); //La première qui finir
```

```
Promise.resolve(val); // forcer la valeur
Promise.reject(new Error())); //forcer l''erreur
```

Le promesses : exemple

```
const unePromesse = new Promise((resolve,reject) =>{
    if (Math.floor(Math.random() * 2)==0)
        resolve("gagné")
    else
        reject("perdu")
})

unePromesse.then(res=>console.log(res)).catch(res=>console.log(res))
console.log("début")
//début
//perdu ou gagné
```

Le promesses : exemple

```
const jeux = (nombre,plage) =>  new Promise((resolve,reject) =>{
    if (Math.floor(Math.random() * plage)==nombre)
        resolve("gagné")
    else
        reject("perdu")
})

jeux(0,1).then(res=>console.log(res)).catch(res=>console.log(res))
console.log("début")
//début
//gagné
```

Le promesses : exemple

```
const timer = (temps) =>  new Promise((resolve) =>{
    setTimeout(() => resolve("fini après " + temps), temps)
})

timer(10)
    .then(s => {console.log(s); return timer(20)})
    .then(s => {console.log(s)})
```

Le promesses : exemple

```
"use strict"
const timer = (temps) =>  new Promise((resolve) =>{
    setTimeout(() => resolve("fini après " + temps), temps)
})

Promise.race([timer(10),timer(20)]).then(s => console.log(s))
//fini après 10
Promise.any([timer(10),timer(20)]).then(s => console.log(s))
//fini après 10
Promise.all([timer(10),timer(20)]).then(s => console.log(s))
//[ 'fini après 10', 'fini après 20' ]
```

Les fonctions asynchrone

```
function resolveAfter2Seconds() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve('resolved');
    }, 2000);
  });
}

async function asyncCall() {
  console.log('calling');
  const result = await resolveAfter2Seconds();
  console.log(result);
  // expected output: "resolved"
}

asyncCall();
```

Les modules

Un module est un fichier souvant avec l'extension .mjs

```
// Exporter des propriétés individuelles
export let nom1, nom2, …, nomN; // utilisable avec var, const
export let nom1 = …, nom2 = …, …, nomN; // utilisable avec var, const
export function nomFonction(){...}
export class NomClasse {...}

export { import1 as nom1, import2 as nom2, …, nomN } from …;
export { default } from …;

// Export d'une liste de valeur
export { nom1, nom2, …, nomN };

// Renommage des valeurs exportées
export { variable1 as nom1, variable2 as nom2, …, nomN };

// Renommage avec la décomposition pour l'affectation
export const { nom1, nom2: truc } = o;

// Exports par défauts
export default expression;
export default function (…) { … } // fonctionne avec class, function*
export default function nom1(…) { … } // fonctionne avec class, function*
export { nom1 as default, … };

// Agrégation de modules
export * from …;
export { nom1, nom2, …, nomN } from …;
```

Les modules

```
import exportParDefaut from "nom-module";
import * as nom from "nom-module";
import { export } from "nom-module";
import { export as alias } from "nom-module";
import { export1 , export2 } from "nom-module";
import { export1 , export2 as alias2 , [...] } from "nom-module";
import exportParDefaut, { export [ , [...] ] } from "nom-module";
import exportParDefaut, * as nom from "nom-module";
import "nom-module";
import { toto , truc } from "nom-module/chemin/vers/fichier-non-exporte";
let promesse = import("nom-module");
```

Les modules

```
//  user.js
export default class User {
  constructor(name) {
    this.name = name;
  }
}

export function sayHi(user) {
  alert(`Hello, ${user}!`);
}
```

```
// main.js
import {default as User, sayHi} from
'./user.js';

new User('John’);

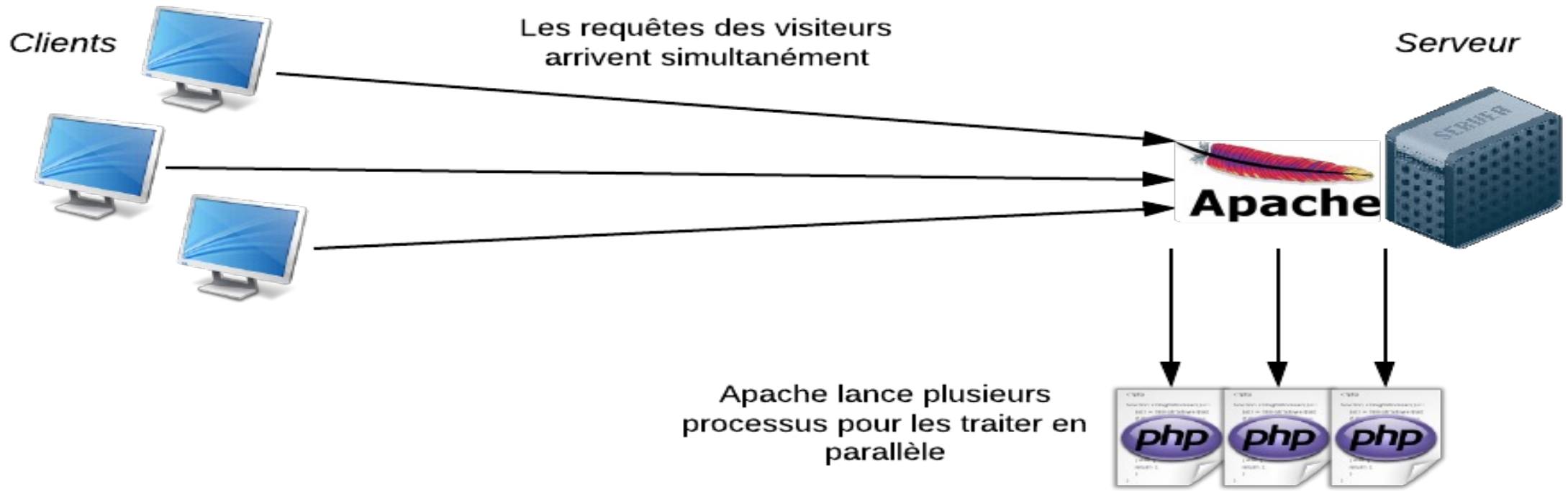
//main.js
import * as user from './user.js';

let User = user.default; // the default
export
new User('John');
```

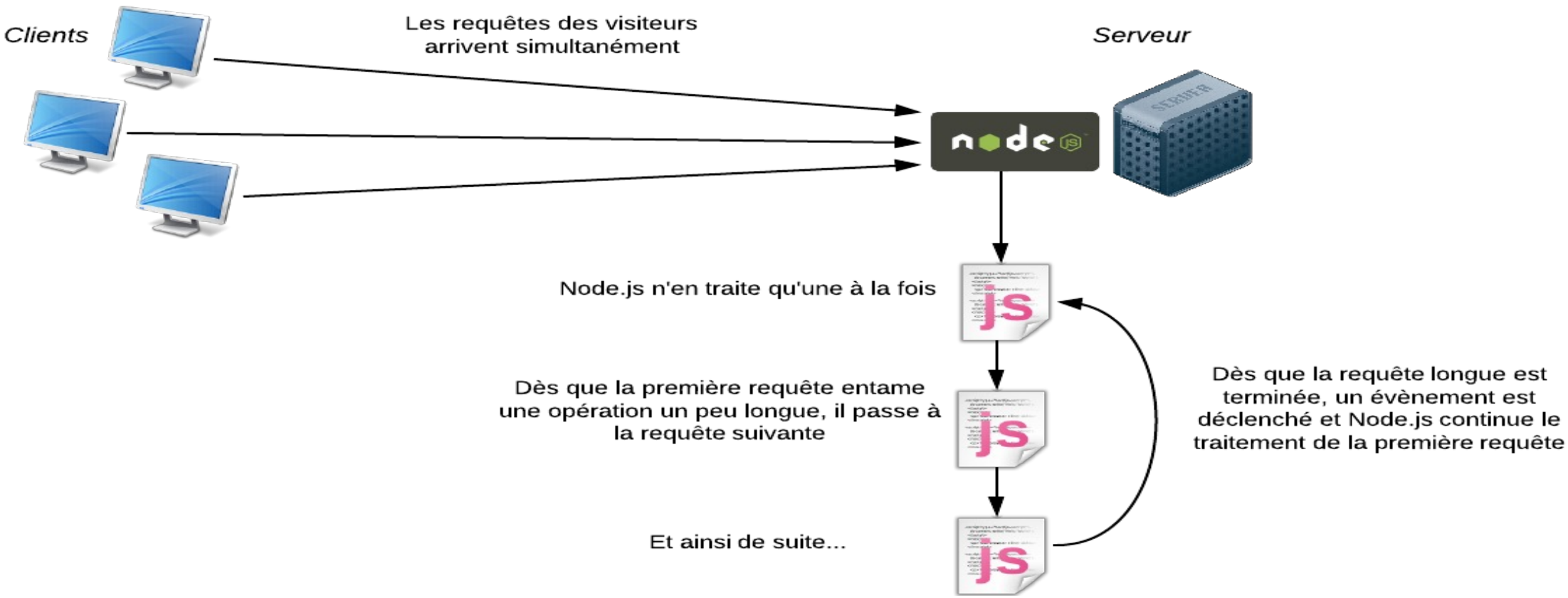
Node JS

- Ce qui suit vient de :
  - http://openclassrooms.com/courses/des-applications-ultra-rapides-avec-node-js

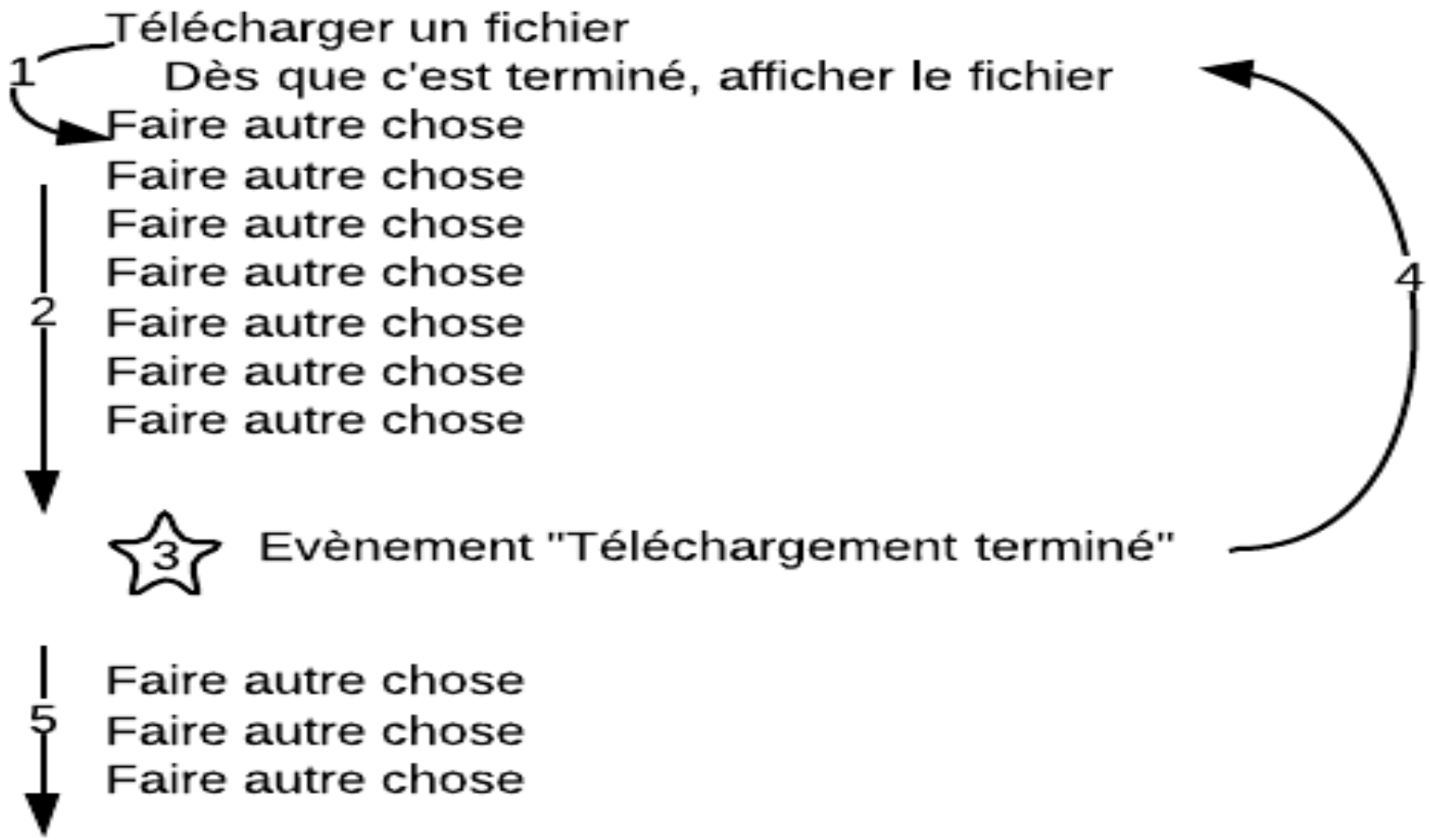
Node JS


Clients
Les requêtes des visiteurs
arrivent simultanément
Serveur
Apache
Apache lance plusieurs
processus pour les traiter en
parallèle
php
php
php

Node JS


Clients
Les requêtes des visiteurs arrivent simultanément
Serveur
Node.js n'en traite qu'une à la fois
Dès que la première requête entame une opération un peu longue, il passe à la requête suivante
Et ainsi de suite...
Dès que la requête longue est terminée, un évènement est déclenché et Node.js continue le traitement de la première requête

Node JS Evenementiel



NodeJS


Client
Serveur
1/ Le client demande la page au serveur
SERVEUR
node
HTML
HTML
HTML
HTML
2/ L'environnement Node.js exécute du JavaScript pour générer le HTML (ou faire d'autres opérations)
3/ Le serveur envoie la page HTML générée au client
4/ Du code JavaScript peut toujours être exécuté par le navigateur du client pour modifier la page HTML

NodeJS :exemple http

```
import http from  'node:http';

const server = http.createServer(function(req, res) {
    res.writeHead(200);
    res.end('Salut tout le monde !');
});
server.listen(8080);
```

NodeJS événements

```
//let EventEmitter = require('events').EventEmitter;
import EventEmitter from 'node:events';

let jeu = new EventEmitter();
//ecoute
jeu.on('gameover', function(message){
    console.log(message);
});
//emission
jeu.emit('gameover', 'Vous avez perdu !');
```

Node JS

Installer des modules

- http://npmjs.org (equivalent apt-get)
- http://nodetoolbox.com/

- npm init

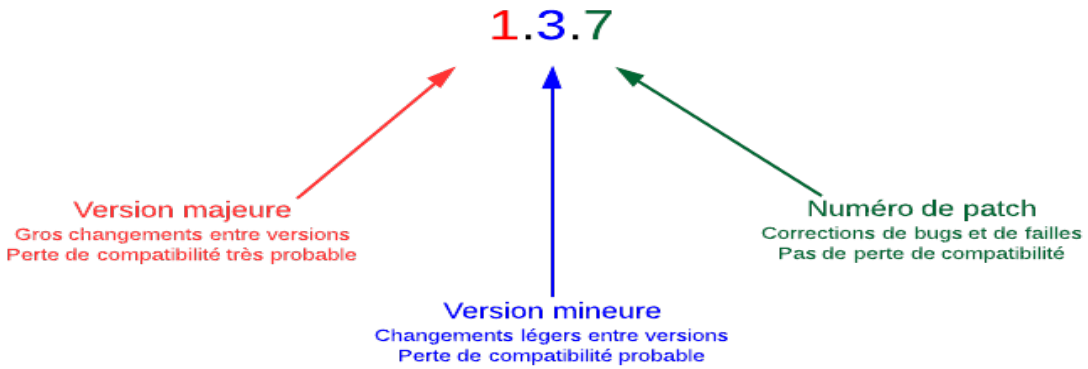
- npm install
- npm install –g (avec sudo)
- npm update

Node JS module

- Les dependances

```
//package.json
{
   "name": "mon-app",
   "version": "0.1.0",
   "dependencies": {
      "markdown": "~0.4"
   }
}
```

- npm update



NodeJS Framework

- Des framework pour faciliter la réalisation d'applications :
  - express.js pour le web
  - hapi.js pour les applications et des services
  - **Socket.io pour les applications connectées « temps-réel »**
  - MetorJS sites web
  - …

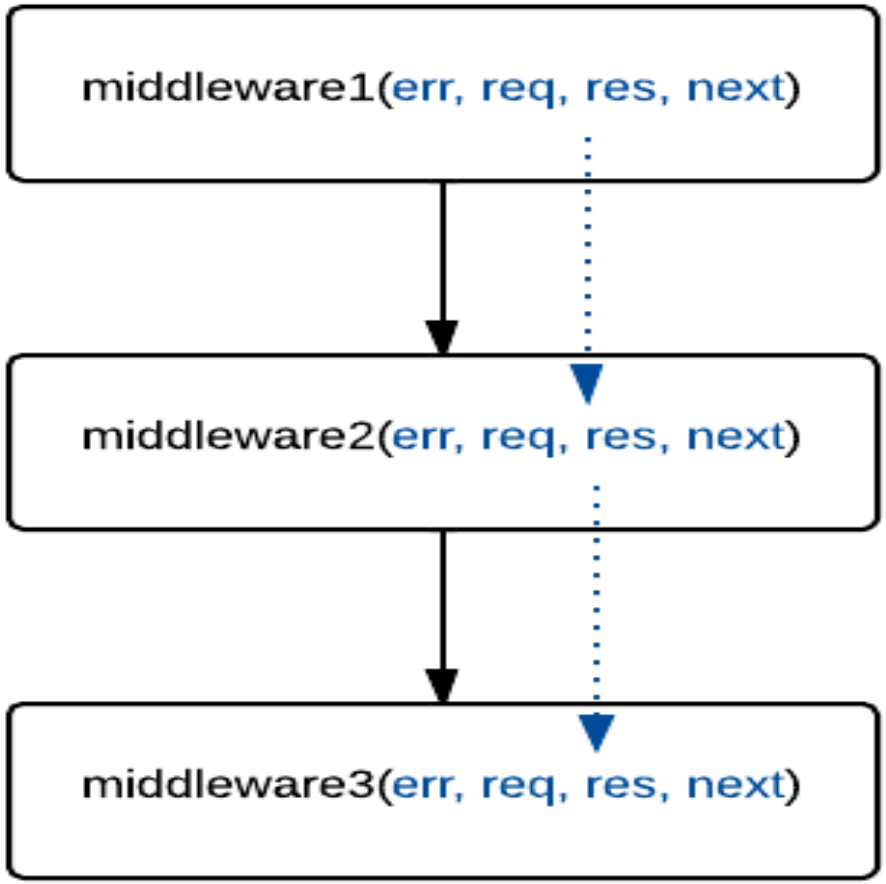
NodeJS framework

- npm install express (celui que nous utiliserons)
  - Gestion des routes, routes dynamiques possibles

Mais d'autres existent

- npm install @hapi/hapi
  - Pour gérer les routes
  - Pour gérer l'authentification
- Nest.js
  - Un cli
    - npm i @nestjs/cli (eviter –g)
    - `./node_modules/.bin/nest`

EJS Midleware



Rest : rappel

Rest : Representational state transfert

Une architecture logicielle qui permet permettent aux systèmes effectuant des requêtes de manipuler des ressources web via leurs représentations textuelles à travers un ensemble d'opérations uniformes et prédéfinies **sans état**.

- Client/serveur
- Sans état => le client doit renvoyer l'ensemble des informations à chaque requête
- Possibilité de mise en cache
- En couche : le client ne sait si il parle à un serveur intermédiaire ou final.
- Interface uniforme :
  - Identification des ressources dans les requêtes (JSON, XML, …)
  - Manipulation des ressources par des représentations (chaque représentation d'un ressource fournie suffisamment d'informations pour la modifier ou la supprimer).
  - Messages auto-descriptifs (le message contient son sens)
  - Hypermédia comme moteur d'état de l'application (**HATEOAS**)
- Avec code à la demande (facultative)

Rest et Web (source wikipédia)

URL	GET	PUT	POST	DELETE
Collection, telle que http://api.exemple.com/ressources/	**Récupère** les URI et éventuellement d'autres détails des éléments de la collection.	**Remplace** la collection par une autre collection.	**Crée** un nouvel élément dans la collection. L'URI du nouvel élément est attribué automatiquement et retourné par cette opération[9].	**Supprime**la collection.
Élément, tel que http://api.exemple.com/ressources/item17	**Récupère** une représentation de l'élément sélectionné de la collection, exprimée dans un type de médiasapproprié.	**Remplace**l'élément sélectionné de la collection, ou s'il n'existe pas, le **crée**.	**Crée** un nouvel élément dans l'élément sélectionné de la collection comme si l'élément sélectionné était lui-même une collection[9]. Cette opération n'est généralement pas utilisée.	**Supprime**l'élément sélectionné de la collection.

Rem : il existe également PATCH pour des modifications partielles

Un exemple d’API

- Lister les membres

GET /api/membre

- Lister un membre (:pseudo du membre recherche)

GET /api/membre/:pseudo

- Créer un membre (:pseudo du membre, :password du membre, :email du membre)

POST /api/membre
BODY
pseudo=:pseudo
password=:password
email=:email

- Modifier un membre (:pseudo du membre ne peut-être modifie, :password nouveau password, :email nouvel email)

  PUT
  /api/membre
  BODY
  pseudo=:pseudo
  password=:password
  email=:email

- Supprimer un membre (:pseudo du membre a supprimer)

  DEL
  /api/membre/:pseudo

Une implémentation avec express

```
import express from 'express';

const app = express()


//Une route
app.get('/about', (req, res) => {
    console.log('url',req.url)
    console.log('params',req.params)
    console.log('query',req.query)
    return res.send(`/about Page : url:
        ${req.url} params ${JSON.stringify(req.params)} query ${JSON.stringify(req.query)}`)
})


app.listen(3000, () =>
    console.log('Example app listening on port 3000!'),
)
```

Exemple de midleware

```
import express from 'express';

const app = express();

app.listen(3000, () =>
    console.log('Example app listening on port 3000!'),
);

const logger  = (req, res, next) => {
    console.log(req.url)
    console.log(req.params)
    console.log(req.query)
    next()
}
app.use(logger)

app.get('/about', (req, res) => {
    return res.send('About Page')
})
```

Server Monolithique en mémoire (modèle)

```
class User {
    login
    password
    constructor(obj) {
        Object.assign(this,obj)
    }
}
class Users {
    users = []
}

//pas de bd uniquement en mémoire
const model = new Users()
```

Server Monolithique en mémoire (DAO)

```
const userDao = {
    //tous les utilisteurs
    findAll : () => model.users,

    //ajout un utilisateur
    //renvoie l'utilisateur ajouté ou null sinon
    findByLogin : (login) => {
        const users = model.users.filter((user)=>user.login==login)
        return users.length!=0 ? users[0]:null
    },


...
```

Server Monolithique en mémoire (Serveur)

```
const init = async () => {
   //Un serveur qui écoute sur le port 3000
   const app = express();
   app.listen(3000, () =>
      console.log('Example app listening on port 3000!'),
   );
   app.use(express.json())
```

…

Server Monolithique en mémoire (Route)

- app.post('/user',(req,res) => {
```
        console.log(req.body)
        const userToAdd = req.body
        const user = userDao.add(userToAdd)
        if (user != null)
            res.status(201).send(user)
        else
            res.status(400).send({message: 'already exist'})
    })
```